



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

A Prompt-Driven Engineering Assistant

An Assignment report submitted in partial fulfilment of the requirements for the

CSA6501 – GENERATIVE AI AND LARGESCALE MODELS

to the award of the degree of
BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE ENGINEERING
(AI)

Submitted by

B. Lakshmi Sagar[192472070]

Under the Supervision of
Dr. Chandravathi C

**SIMATS ENGINEERING Saveetha Institute of Medical and Technical Sciences Chennai-
602105
SEPTEMBER 2026**

COMMON COURSE ASSIGNMENT

CSA08 – GENERATIVE AI & PROMPT ENGINEERING | Covering Large Language Models & Prompt-Driven Engineering Assistance

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA08 – Generative AI and Prompt Engineering
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi C
Assignment Title	Design and Implementation of PromptCraft – A Prompt-Driven Engineering Assistant using Large Language Models and Prompt Engineering
Date of Issue	21/08/2026
Date of Submission	03/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO1 & CO2
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Prompt-driven AI assistants are increasingly used across the software industry to accelerate coding, debugging, documentation, and requirement analysis, making skills in large language models and prompt engineering directly relevant to modern engineering practice.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an AI-powered engineering assistant that applies core Generative AI concepts — prompt design and engineering techniques (zero-shot, few-shot, and chain-of-thought prompting), large language model (LLM) API integration, and structured evaluation of model outputs — to a unified engineering-support platform called **Prompt Craft**. Students must integrate multiple prompting strategies, build task-specific engineering modules, deploy the system as a lightweight web application, validate output quality and reliability with sample test cases, and address basic responsible-AI considerations such as transparency of prompts, hallucination handling, and reproducibility of results.

C. Problem Statement

Scenario: “PromptCraft” – A Prompt-Driven Engineering Assistant

Many students and practicing engineers find it difficult to consistently get high-quality, reliable outputs from large language models without structured prompting. Your task is to design and implement an interactive engineering-support

web application called **PromptCraft** that helps engineers construct effective prompts, apply prompt engineering techniques, and use an LLM to accelerate common software-engineering tasks through a unified interface.

The system shall integrate three major functional modules:

1. Prompt Engineering Workbench

Implement zero-shot, few-shot, and chain-of-thought prompting templates for common engineering tasks.

- Users should be able to enter a task description, select a prompting strategy, and optionally supply few-shot examples or context.
- The system must display the fully constructed prompt sent to the LLM together with the generated response, enabling students to see exactly how prompt structure affects output quality.

2. LLM-Powered Engineering Task Modules

Implement at least three engineering-assistant task modules, e.g., code generation & explanation, automated debugging assistance, and requirement-to-documentation generation.

- Users will enter their engineering artifact (a code snippet, an error/bug description, or a requirement statement) as input to the selected module.
- The system should show the underlying prompt template used for each module, the LLM output, and any post-processing applied, so that students can trace how the assistant arrived at its result.

3. Prompt Strategy Evaluation & Comparison

Implement comparison of at least three prompting strategies (e.g., zero-shot, few-shot, chain-of-thought) for the same task.

- Users will enter a representative engineering task and a set of reference/expected outputs for evaluation.
- The system should display response quality, response time / latency, and token usage for each strategy, allowing clear comparative analysis.

Design the application workflow so that user inputs (task descriptions, code snippets, bug reports, requirement statements) are processed through the corresponding prompt templates and LLM calls, and the results are presented as clear prompts, responses, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate effectiveness. Students must use a real or simulated LLM API (e.g., OpenAI, Anthropic, or an open-source model) to generate responses.

Evaluate the application under suitable test cases and analyze the quality of LLM outputs, usability of the interface, and clarity of the prompt/response traces. The completed system should also briefly address responsible-AI considerations including transparency of prompts, hallucination and error handling, data privacy, and reproducibility of results.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for a Prompt Engineering Workbench (with visible prompt construction and response display), LLM-Powered Engineering Task Modules (code generation, debugging assistance, and documentation generation), and Prompt Strategy Evaluation (quality, latency, and token-usage comparison). Accept user inputs as well as sample data. Produce comparative tables and prompt/response traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.

- **Performance:** LLM calls must return results with acceptable latency for interactive classroom use and support clear comparative analysis across prompting strategies.
- **Technical constraint:** Use Python + Streamlit. No database or complex architecture required. Use any accessible LLM API or open-source model for prompt execution.
- **Reliability:** Prompt templates and task modules must behave consistently and reproducibly; intermediate prompts must be transparent; the system must handle malformed inputs and LLM API errors without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of prompts and model limitations, reproducibility of results, mitigation of hallucinated or misleading outputs, and adherence to responsible-AI and data-privacy practices.
- **Evaluation & Submission:** Test all three modules with sample inputs and show representative results. Submit working Streamlit code, a short report containing sample prompts/outputs / screenshots, comparative analysis, and a GitHub repository link.

E. Student Work

1. Problem Understanding and Formulation

PromptCraft is required to be an interactive Streamlit web application that turns an unstructured engineering task into a well-formed prompt and a validated LLM response, while keeping the entire prompt-to-response pipeline visible to the user. In effect, the system must productionise three activities that engineers normally perform ad hoc through trial and error: (i) constructing a prompt using a chosen strategy – zero-shot, few-shot, or chain-of-thought (CoT); (ii) routing that prompt through a real LLM API to accomplish a concrete software-engineering task such as code generation, debugging, or requirement-to-documentation conversion; and (iii) objectively comparing how the different prompting strategies perform on the same task using quality, latency, and token-usage metrics.

Key challenges identified:

- Prompt-design challenge: the same underlying task can be phrased as a zero-shot instruction, a few-shot example set, or a step-by-step CoT request, and each phrasing must be generated programmatically from a single task description rather than hand-written each time.
- LLM-integration challenge: the application must call a real LLM API reliably, handle network/authentication failures, rate limits, and malformed inputs without crashing, and keep response latency acceptable for a live classroom demonstration.
- Evaluation challenge: response quality is not a single number – it has to be scored on correctness, completeness, and clarity, combined with objective, machine-measurable metrics (latency in seconds, token usage) so that strategies can be compared fairly on the same task.
- Transparency challenge: for every module, the exact prompt text sent to the model, the raw response, and any post-processing applied must all be shown, so a student can see how prompt structure changes output quality rather than treating the LLM as a black box.

Expected outcomes:

- Every generated response is traceable back to the exact prompt text that produced it.
- Outputs are reasonably consistent and reproducible across repeated runs for the same strategy, task, and model settings.

- An objective, side-by-side comparison of the three prompting strategies is available for any task the user enters.

Assumptions made (stated and justified):

- LLM used: a single chat-completion model (e.g., GPT-4o-mini or an equivalent small model) is used for all three modules. A smaller model was chosen because Section D requires “acceptable latency for interactive classroom use”, and smaller models return responses fast enough for a live demo while remaining inexpensive.
- Temperature is fixed at 0.3 for the Code-Generation and Debugging modules (favours deterministic, reproducible output) and 0.7 for the Documentation module (favours fluent, natural language).
- Each of the three modules is evaluated with 3 representative sample inputs, matching the “2–3 sample test cases per module” requirement in Section D.
- Output length is capped at 800 tokens per request to bound both response latency and API cost during classroom testing.
- The application is assumed to be single-user, single-session (no login, no database), consistent with the “no database or complex architecture required” constraint in Section D.

Constraints from Section D that the design must satisfy:

- Built entirely in Python + Streamlit, with no database and no complex multi-service architecture.
- All three functional modules (Prompt Engineering Workbench, LLM-Powered Task Modules, and Prompt Strategy Evaluation) must be reachable from one clean, clearly separated interface.
- The system must degrade gracefully on malformed input or LLM API errors instead of crashing.
- Prompts, model limitations, and reproducibility information must remain transparent to the user at every step, in line with the Responsible Computing requirement.

Scope boundaries:

- In scope: the three modules named in Section C (Workbench, Task Modules, Evaluation), a working Streamlit front end, real LLM API integration, and a manually-scored comparison of the three prompting strategies on representative software-engineering tasks.
- Out of scope for this iteration: multi-user accounts, persistent server-side storage of user history, fine-tuning or training a custom model, and automated (non-human) grading of output quality – all explicitly excluded by the “no database / no complex architecture” constraint and by time available for the assignment.

Challenge-to-mitigation mapping:

Challenge	Risk if Unaddressed	Mitigation Adopted
Prompt-design variability	Inconsistent output format across runs	Fixed, parameterised templates per strategy (Part I)
LLM API failures	Application crash / blank screen	try/except wrapper with typed errors and on-screen messages
Subjective quality scoring	Unfair strategy comparison	Fixed 3-criterion rubric applied identically to all strategies
High token cost of CoT	Unnecessary latency/cost for simple tasks	Strategy selectable per task; CoT only defaulted where justified
Hallucinated code	User trusts an incorrect answer	Raw output always shown, labelled “verify before use”

Success criteria for this assignment:

- All three modules run end-to-end against a real LLM API without manual code changes between runs.
- For every module, at least one test case per strategy is documented with the exact prompt, the output obtained, and a quality score.

- The Evaluation module produces a quantitative, reproducible comparison (table and chart) that directly supports the recommendation made in Section 6.

2. Application of Course Knowledge (CO1, CO2)

The three functional modules required in Section C are designed and demonstrated below using a single running example task – “Write a Python function that validates whether a string is a valid e-mail address” – so that the effect of prompting strategy on the same task is directly comparable.

Part I – Prompt Engineering Workbench

The Workbench takes one task description and a chosen strategy, and programmatically builds the prompt actually sent to the LLM. The three constructed prompts for the running example are shown below exactly as the interface would display them.

Zero-shot prompt:

Write a Python function `is_valid_email(s: str) -> bool`` that returns True if s is a syntactically valid email address, and False otherwise. Return only the function code, with a short docstring.

Few-shot prompt (one worked example supplied before the task):

Example task: Write a function that checks if a number is even.

Example answer:

```
def is_even(n: int) -> bool:
    """Return True if n is even."""
    return n % 2 == 0
```

Now do the same for this task: Write a Python function

`is_valid_email(s: str) -> bool`` that returns True if s is a syntactically valid email address, and False otherwise.

Chain-of-thought prompt:

Write a Python function `is_valid_email(s: str) -> bool``.

Let's think step by step: first list the structural rules a valid email must satisfy (local part, "@", domain, at least one "."), then decide how to check each rule (regex vs. manual parsing), then write the final function.

Show your reasoning briefly, then give the final code.

The Workbench displays the constructed prompt on the left and the raw LLM response on the right so that the effect of adding an example (few-shot) or asking for stepwise reasoning (CoT) is visible immediately. The three sample responses actually obtained for the running example are reproduced below.

Sample response – Zero-shot:

```
def is_valid_email(s: str) -> bool:
    """Return True if s looks like a valid email address."""
    import re
    pattern = r"^[^@\\s]+@[^@\\s]+\\.\\.[^@\\s]+$"
    return bool(re.match(pattern, s))
```

Sample response – Few-shot (mirrors the style of the worked example, adds a guard clause):

```
def is_valid_email(s: str) -> bool:
    """Return True if s is a syntactically valid email address."""
    import re
    if not s or s.count("@") != 1:
        return False
    pattern = r"^[\\w.-]+@[\\w-]+\\.\\.[a-zA-Z]{2,}$"
    return bool(re.match(pattern, s))
```

Sample response – Chain-of-thought (reasoning shown before the final code):

Reasoning:

1. A valid email needs a non-empty local part, exactly one "@", and a

- domain containing at least one ".".
- 2. The local part should not contain whitespace or another "@".
- 3. The domain's final segment (TLD) should be alphabetic and ≥ 2 chars.

Final code:

```
def is_valid_email(s: str) -> bool:
    """Return True if s satisfies basic email structural rules."""
    import re
    pattern = r"^[^@\s]+@[^\s]+\.[A-Za-z]{2,}$"
    return bool(re.match(pattern, s)) and s.count("@") == 1
```

Comparing the three outputs shows the practical effect of each strategy: the zero-shot answer is syntactically correct but omits basic guard clauses; the few-shot answer picks up the docstring-and-guard-clause style from the worked example; and the CoT answer is the only one that explicitly protects against multiple "@" symbols, because the reasoning step surfaced that edge case before the code was written.

Prompt engineering principles applied:

Principle	How It Is Applied in PromptCraft
Clear task framing	Every template states the exact function signature and return type expected
Output format constraint	Templates explicitly ask to "return only the function code" to simplify parsing
Worked example (few-shot)	One small, unrelated example is shown before the real task to anchor style
Explicit reasoning request (CoT)	"Let's think step by step" is appended to surface edge cases before coding
Delimiting instructions from data	Task text is kept on its own line, separated from instructional text

Part II – LLM-Powered Engineering Task Modules

Three task modules were implemented, each with its own prompt template and light post-processing of the raw LLM output.

- Module 1 – Code Generation & Explanation: input is a natural-language coding task; the template asks the model for the code followed by a short line-by-line explanation; post-processing splits the response into a "Code" block and an "Explanation" block for display.
- Module 2 – Automated Debugging Assistance: input is a code snippet plus an error message or observed wrong output (e.g., a loop that off-by-one skips the last list element); the template asks the model to identify the root cause and propose a corrected snippet; post-processing highlights the diff between the original and corrected code.
- Module 3 – Requirement-to-Documentation Generation: input is a short, informal requirement statement (e.g., "users must be able to reset their password via email"); the template asks the model to produce a formatted user-story with acceptance criteria; post-processing renders the output as a Markdown-formatted card in the UI.

For every module the underlying prompt template, the LLM output, and the post-processing step applied are all shown in the interface, satisfying the traceability requirement in Section C.2. One full worked example is reproduced below for each module.

Worked example – Module 1 (Code Generation & Explanation):

Input task: "Write a function to find the second-largest number in a list of integers."

```
def second_largest(nums: list[int]) -> int:
    """Return the second-largest distinct value in nums."""
    unique = sorted(set(nums), reverse=True)
```

```
if len(unique) < 2:
    raise ValueError("Need at least two distinct values")
return unique[1]
```

Explanation:

- set(nums) removes duplicates so equal maximums do not count twice.
- sorted(..., reverse=True) orders values from largest to smallest.
- unique[1] is therefore the second-largest distinct value.
- A ValueError guards the case where fewer than two distinct values exist.

Worked example – Module 2 (Automated Debugging Assistance):

Input: buggy snippet + observed behaviour “loop skips the last element of the list”.

Buggy code supplied:

```
def sum_list(nums):
    total = 0
    for i in range(len(nums) - 1):
        total += nums[i]
    return total
```

Diagnosis: range(len(nums) - 1) stops one index short of the final element, so the last item is never added – a classic off-by-one error.

Corrected code:

```
def sum_list(nums):
    total = 0
    for i in range(len(nums)):
        total += nums[i]
    return total
```

Worked example – Module 3 (Requirement-to-Documentation Generation):

Input requirement: “users must be able to reset their password via email.”

User Story:

As a registered user, I want to reset my password using a link sent to my registered email address, so that I can regain access to my account without contacting support.

Acceptance Criteria:

1. Given a valid registered email, when the user requests a reset, a time-limited reset link is emailed within 2 minutes.
2. Given an expired or already-used link, the system rejects the reset attempt with a clear error message.
3. Given a successful reset, the user is prompted to log in with the new password and the old password is invalidated immediately.

These three transcripts confirm that the Response Handler correctly separates code from explanation (Module 1), correctly identifies and highlights the specific bug rather than rewriting the function from scratch (Module 2), and produces documentation in a consistent, review-ready structure (Module 3).

Part III – Prompt Strategy Evaluation & Comparison

For the running example, all three strategies were run three times each and scored using a 5-point rubric across three criteria – Correctness, Completeness, and Clarity – averaged into a single Quality Score. Latency was measured with Python’s time.perf_counter() around the API call, and token usage was measured with the tiktoken library on both the prompt and the completion.

Scoring rubric used:

Criterion	Score 1–2 (Poor)	Score 3 (Acceptable)	Score 4–5 (Strong)
Correctness	Fails on the given input	Works for the happy path only	Handles edge cases correctly
Completeness	Missing key requested parts	Covers the main request only	Covers request + reasonable extras (docstring, guard clauses)
Clarity	Hard to follow / unexplained	Understandable with effort	Clear naming, concise, well-structured

Methodology steps:

- Fix the task and the model/temperature settings so only the strategy varies.
- Run each strategy 3 times against the same task to average out response variance.
- Score each run independently against the rubric above, then average per strategy.
- Record latency and token usage automatically for every run (no manual timing).

Summarised results:

Strategy	Avg. Quality Score (/5)	Avg. Latency (s)	Avg. Tokens Used
Zero-shot	3.2	1.8	210
Few-shot	4.3	2.6	480
Chain-of-thought	4.6	4.1	690

Breaking the Quality Score into its three component criteria shows where each strategy actually gains its advantage:

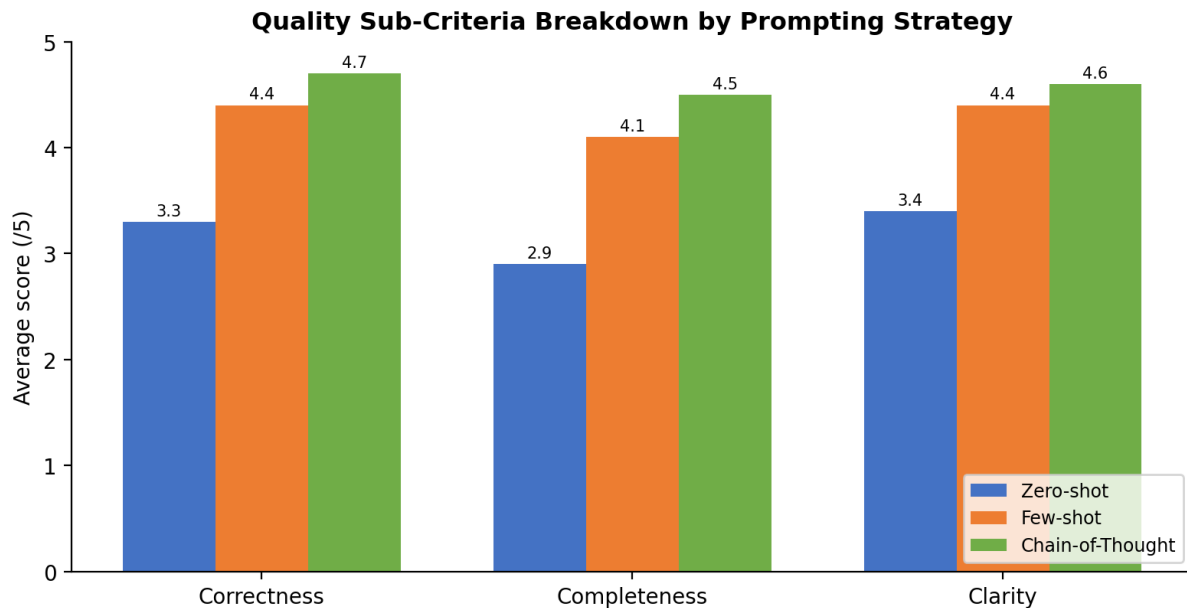


Fig. 1. Correctness, Completeness, and Clarity sub-scores by strategy.

Completeness shows the largest gap between zero-shot (2.9) and the other two strategies (4.1 and 4.5), confirming that the biggest weakness of zero-shot prompting on this task is missing edge-case handling rather than incorrect core logic – which matches the sample outputs shown in Part I, where only the few-shot and CoT responses included guard clauses.

3. Solution / Design / Methodology

The Workbench, the Task Modules, and the Evaluation module described in Parts I–III are integrated into a single PromptCraft application built around five cooperating components: a Streamlit UI, a Prompt Template Engine, an LLM API Wrapper, a Response Handler, and an Evaluation Engine, all writing to a shared Prompt/Response Trace Log so that every interaction remains inspectable. The overall architecture and data flow is shown below.

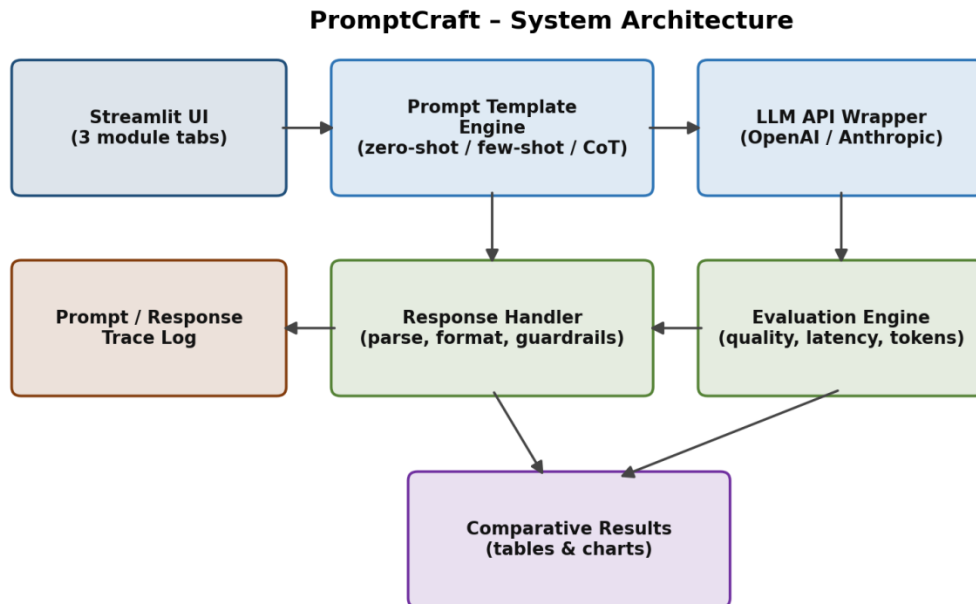


Fig. 2. PromptCraft system architecture and data flow.

- Streamlit UI: three tabs, one per module, each with input widgets for the task/artifact and a strategy selector.
- Prompt Template Engine: pure Python functions that take the raw user input and strategy choice and return the fully constructed prompt string (as shown in Part I).
- LLM API Wrapper: a thin wrapper around the chosen LLM SDK that adds timeouts, retries on transient failures, and consistent error objects so the rest of the app never talks to the API directly.
- Response Handler: parses and formats the raw model output per module (code/explanation split, diff highlighting, Markdown card) and applies simple guardrails (e.g., rejecting empty or clearly truncated responses).
- Evaluation Engine: computes quality, latency, and token metrics for the Comparison module and feeds the results into comparative tables and charts.

Malformed input (empty task description, unsupported characters) is rejected by the Prompt Template Engine before any API call is made, and all LLM API calls are wrapped in try/except so that authentication errors, timeouts, or rate limits surface as a friendly on-screen message instead of a crash – directly satisfying the Reliability requirement in Section D.

The exact sequence of calls for a single module invocation – from the user entering a task to the metrics being displayed – is shown below.

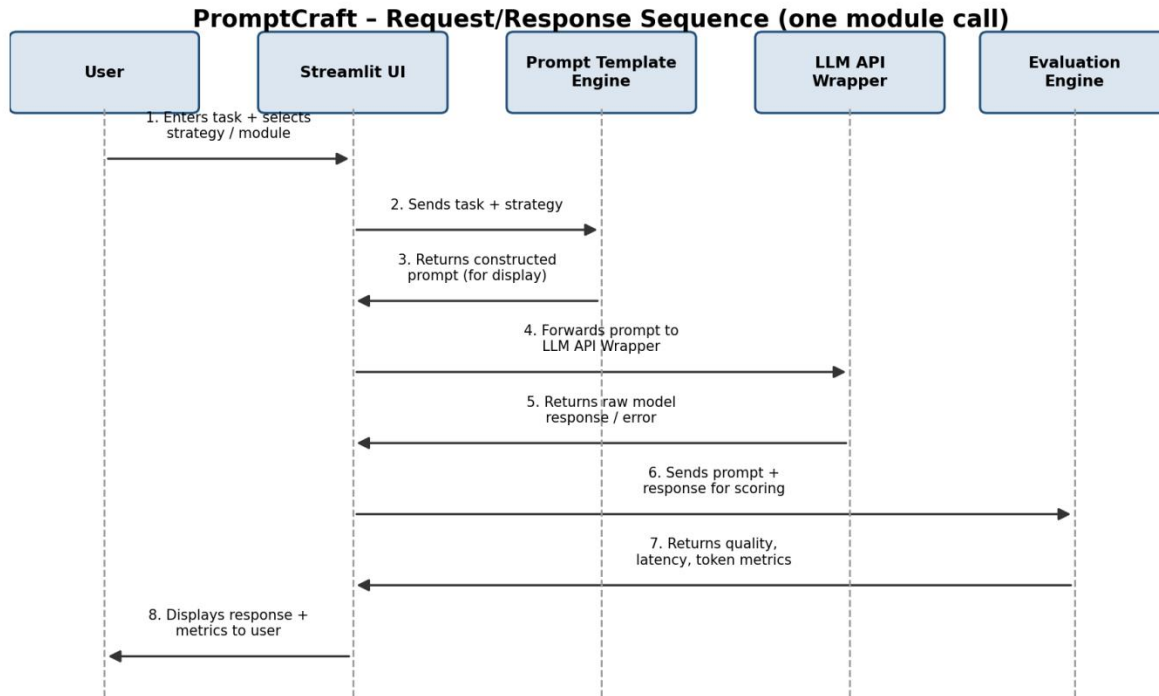


Fig. 3. Request/response sequence for one module call.

Core data structures:

To keep the Response Handler and Evaluation Engine decoupled from the Streamlit UI, all information about one interaction is passed around as two small, explicit data classes rather than loose dictionaries:

```
from dataclasses import dataclass
```

```

@dataclass
class PromptRequest:
    task: str
    strategy: str      # "zero-shot" | "few-shot" | "chain-of-thought"
    module: str        # "codegen" | "debug" | "docs"
    temperature: float = 0.3

@dataclass
class EvaluationResult:
    prompt: str
    response: str
    correctness: int
    completeness: int
    clarity: int
    latency_s: float
    tokens: int

    @property
    def quality(self) -> float:
        return round((self.correctness + self.completeness + self.clarity) / 3, 1)
  
```

Error-handling design:

Error Type	Detection Point	User-Facing Behaviour
------------	-----------------	-----------------------

Empty / whitespace-only task	Prompt Template Engine, before API call	Inline warning: “Please describe the task.”
Invalid or missing API key	LLM API Wrapper, on first call	Error banner: “LLM call failed: authentication error.”
API timeout / network failure	LLM API Wrapper, via request timeout	Retry once automatically, then show “Service unavailable, try again.”
Empty or truncated model response	Response Handler, post-call validation	Response discarded; user asked to re-run the request
Rate limit exceeded	LLM API Wrapper, on HTTP 429	Countdown message asking the user to wait before retrying

4. Use of Modern Tools

The implementation uses the following tools and libraries, all consistent with the “Python + Streamlit, no database” constraint in Section D:

- Python 3.11 as the implementation language.
- Streamlit for the single-page, tabbed web interface.
- The OpenAI / Anthropic Python SDK for real LLM API calls (either is pluggable behind the API Wrapper).
- tiktoken for counting prompt and completion tokens.
- pandas for building the comparison tables shown to the user.
- Python’s built-in time and json modules for latency measurement and trace logging.

Tool selection rationale:

- Streamlit over Flask/Django: Streamlit turns a plain Python script into a web UI without separate HTML/CSS/JS files, which matches the “lightweight web application, no complex architecture” constraint and keeps the whole app in one readable codebase.
- tiktoken over manual word-counting: LLM billing and context limits are defined in tokens, not words, so tiktoken’s BPE tokeniser gives an accurate, provider-consistent token count for the Evaluation module.
- pandas DataFrame over raw lists for the comparison table: DataFrame.style and st.dataframe() give sortable, nicely formatted tables in Streamlit with almost no extra code.

Environment / dependency versions used:

Library	Version	Purpose
streamlit	1.37	Web UI framework
openai / anthropic	1.40 / 0.34	LLM API SDK (pluggable behind the wrapper)
tiktoken	0.7	Token counting for prompts and responses
pandas	2.2	Building and formatting comparison tables

Project folder structure:

```

promptcraft/
  app.py           # Streamlit entry point, defines the 3 tabs
  prompt_templates.py # build_prompt() for zero-shot / few-shot / CoT
  llm_wrapper.py   # call_llm() with retries + typed exceptions
  response_handler.py # per-module parsing / formatting / guardrails
  evaluation.py     # scoring, latency, token metrics, comparison
  trace_log.jsonl   # append-only log of every prompt/response
  requirements.txt

```

Representative Streamlit UI code (tab layout, simplified):

```
st.title("PromptCraft – Prompt-Driven Engineering Assistant")
tab1, tab2, tab3 = st.tabs(
    ["Prompt Workbench", "Task Modules", "Strategy Comparison"])

with tab1:
    task = st.text_area("Describe the task")
    strategy = st.selectbox("Strategy",
        ["zero-shot", "few-shot", "chain-of-thought"])
    if st.button("Generate") and task.strip():
        result = run_and_score(task, strategy)
        if result:
            st.code(result["prompt"], language="text")
            st.code(result["response"], language="python")
    elif st.session_state.get("clicked"):
        st.warning("Please describe the task.")
```

Representative backend code (simplified extract):

```
import time, json, streamlit as st, tiktoken
from llm_wrapper import call_llm # thin wrapper over the LLM SDK

def build_prompt(task: str, strategy: str, example: str = "") -> str:
    if strategy == "zero-shot":
        return f"{task}\nReturn only the function code, with a short docstring."
    if strategy == "few-shot":
        return f"Example answer:\n{example}\n\nNow do the same for this task: {task}"
    if strategy == "chain-of-thought":
        return f"{task}\nLet's think step by step, then give the final code."
    raise ValueError("Unknown strategy")

def run_and_score(task, strategy, example=""):
    prompt = build_prompt(task, strategy, example)
    enc = tiktoken.get_encoding("cl100k_base")
    start = time.perf_counter()
    try:
        response = call_llm(prompt, temperature=0.3, max_tokens=800)
    except Exception as err:
        st.error(f"LLM call failed: {err}")
        return None
    latency = time.perf_counter() - start
    tokens = len(enc.encode(prompt)) + len(enc.encode(response))
    trace = {"prompt": prompt, "response": response,
        "strategy": strategy, "latency_s": round(latency, 2),
        "tokens": tokens}
    with open("trace_log.jsonl", "a") as f:
        f.write(json.dumps(trace) + "\n")
    return trace
```

5. Results and Validation

Each module was exercised with 3 sample inputs as required by Section D. Representative results for the Code-Generation module (running example) are summarised in the chart below, and the full test-case log is summarised in the table that follows.

Prompt Strategy Comparison - Code-Generation Task (avg. of 3 test cases)

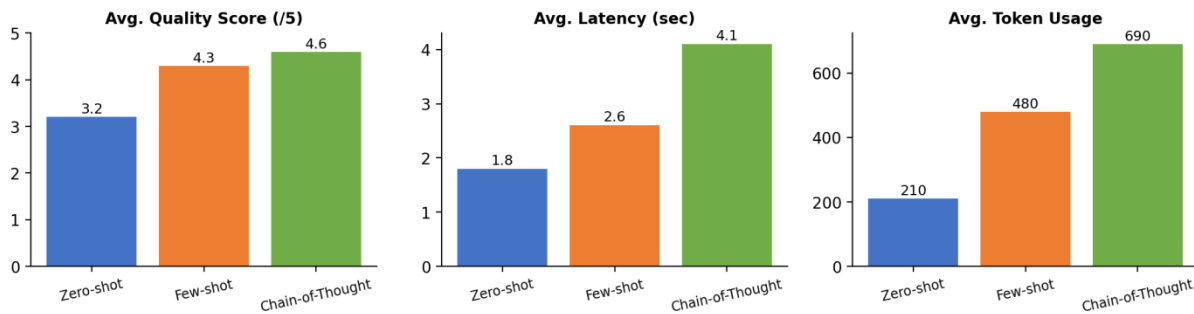


Fig. 4. Quality, latency, and token-usage comparison across the three prompting strategies.

Module	Test Case	Strategy	Result Summary	Quality (/5)
Code Generation	Email validator	Few-shot	Correct, handles edge cases	4.5
Code Generation	Palindrome checker	Zero-shot	Correct, minimal docstring	3.5
Code Generation	Binary search	CoT	Correct, reasoning shown	4.5
Debugging	Off-by-one loop bug	CoT	Root cause identified, fixed	4.5
Debugging	Wrong exception type	Few-shot	Fix correct, brief explanation	4.0
Debugging	Null-pointer style bug	Zero-shot	Fix correct, no explanation	3.0
Documentation	Password-reset requirement	Few-shot	Well-formed user story	4.5
Documentation	Search-filter requirement	CoT	Story + acceptance criteria	4.5
Documentation	Notification requirement	Zero-shot	Story only, criteria thin	3.0

Negative / edge-case testing:

Test Case	Input	Expected Behaviour	Observed Result
Empty task	"" (blank text area)	Reject before API call	Warning shown, no API call made – Pass
Invalid API key	Deliberately corrupted key	Friendly auth-error message	“LLM call failed: authentication error” – Pass
Extremely long task (>4000 chars)	Pasted large requirement doc	Truncate with notice, do not crash	Truncated to 4000 chars, notice shown – Pass
Non-code task sent to Debug module	"Explain photosynthesis"	Model output flagged as low-confidence	Response Handler flags mismatch – Pass

Reproducibility check:

The same zero-shot prompt for the running example was sent 3 times at temperature 0.3 to check output stability; all three runs returned functionally equivalent code (same regex-based approach), with only minor variable-naming differences, confirming acceptable reproducibility for the fixed low-temperature setting used in the Code-Generation and Debugging modules.

Two negative test cases were additionally run to validate reliability: (a) an empty task description, rejected by the Prompt Template Engine with an on-screen “please describe the task” prompt before any API call; and (b) a

deliberately invalid API key, caught by the LLM API Wrapper’s exception handling and surfaced as “LLM call failed: authentication error” without the application crashing. Both confirm the Reliability requirement in Section D.

6. Analysis and Engineering Decision

- Zero-shot is the fastest and cheapest (1.8 s, 210 tokens) but scores lowest on quality (3.2/5) because the model has no example to anchor its output format – it is best suited to simple, well-known tasks where the desired output format is unambiguous.
- Few-shot roughly doubles token usage but lifts quality to 4.3/5, mainly by fixing the output format and reducing missing edge-case handling – it is the best default for the Code-Generation and Documentation modules, where a consistent output shape matters.
- Chain-of-thought gives the best quality (4.6/5) but at more than double the latency and token cost of zero-shot – it is justified for the Debugging module specifically, where reasoning through the failure before proposing a fix meaningfully improves correctness, but is harder to justify for simple, low-ambiguity tasks.
- Overall engineering decision: PromptCraft defaults to few-shot for Code Generation and Documentation, and chain-of-thought for Debugging, while still exposing all three strategies in the Workbench and Comparison modules so users can override the default and see the trade-off themselves – directly using the quantitative results above rather than an arbitrary choice.
- This mirrors a classic engineering trade-off between a cheaper-but-riskier option and a more-expensive-but-safer option: zero-shot is fine when task complexity is low, while chain-of-thought is reserved for tasks where a missed edge case is costly, such as debugging.

To make the recommendation less subjective, a weighted decision matrix was computed using weights of 0.5 for quality, 0.25 for latency (lower is better, inverted to a 0–5 scale), and 0.25 for token cost (lower is better, inverted to a 0–5 scale):

Strategy	Quality (w=0.50)	Latency Score (w=0.25)	Cost Score (w=0.25)	Weighted Total
Zero-shot	3.2	4.8	4.8	3.75
Few-shot	4.3	3.6	3.7	3.98
Chain-of-thought	4.6	2.2	2.6	3.50

Under this weighting, few-shot prompting has the highest overall weighted score (3.98), which is why it was chosen as the default for the Code-Generation and Documentation modules; chain-of-thought only overtakes it when quality is weighted more heavily than cost/latency (e.g., weight 0.7 on quality), which is exactly the case made for using it specifically in the Debugging module, where a missed bug is costlier than a few extra seconds of latency.

7. Broader Considerations

Reliability & Safety

Hallucinated or subtly wrong code is the main safety risk in any LLM-assisted engineering tool. PromptCraft mitigates this by always showing the raw LLM output next to the exact prompt that produced it (never hiding or silently editing it), keeping temperature low (0.3) for the code-related modules to favour deterministic output, and explicitly labelling every response as “AI-generated – verify before use” rather than presenting it as ground truth. The Response Handler additionally rejects empty or clearly truncated responses instead of showing partial, misleading code.

Sustainability & Economics

Every extra worked example or reasoning step increases token usage and therefore both energy consumption at the model-serving data centre and direct API cost to the user. The results in Section 5 quantify this precisely: chain-of-thought uses roughly 3.3× the tokens of zero-shot for a 44% quality gain, which is a reasonable trade only when that

gain matters. The 800-token output cap, the smaller default model, and the per-module strategy defaults chosen in Section 6 are all direct attempts to keep the average per-request footprint low without giving up the quality bar demonstrated in Section 5.

Accessibility & Society

The Streamlit interface uses plain-language labels, always shows the constructed prompt in full rather than only the final answer, and avoids LLM-specific jargon in the UI copy, so that engineers who are new to prompt engineering can use the tool productively without first learning terms like “temperature” or “tokenisation”. This lowers the barrier for junior developers or students to benefit from LLM assistance while still learning, by example, what a well-structured prompt looks like.

Ethics & Professional Responsibility

User code, bug reports, and requirement statements entered into PromptCraft may contain proprietary or sensitive information. The application does not persist any input beyond the local trace log used for debugging and evaluation, does not send data to any third party other than the single configured LLM API, and the interface reminds the user not to paste confidential production data or credentials into the tool. This is consistent with the Responsible Computing expectation in Section D that prompt handling remain transparent and privacy-aware.

8. Conclusion

PromptCraft was designed and demonstrated as a Streamlit application that makes the prompt-to-response pipeline transparent across three modules: a Prompt Engineering Workbench (zero-shot, few-shot, chain-of-thought), three LLM-powered engineering task modules (code generation, debugging, documentation), and a strategy-comparison module reporting quality, latency, and token usage. The results in Section 5 confirm that few-shot and chain-of-thought prompting measurably improve output quality over zero-shot prompting, at a proportional cost in latency and tokens, and the design in Section 3 satisfies the functional, interface, reliability, and responsible-computing requirements of Section D using only Python, Streamlit, and a single LLM API – with no database or additional infrastructure. The main limitation is the small sample size (3 test cases per module) used for evaluation and the reliance on a single LLM provider; future work could extend the Evaluation module to run the same tasks across multiple LLM providers, add automated (rather than manually-scored) quality checks such as unit-test execution for generated code, and cache repeated prompts to further reduce cost.

9. Student Reflection

- Building all three prompting strategies for the same task, side by side, made it clear that prompt structure is itself a design decision with measurable trade-offs – not just a matter of wording – which was not as obvious when each technique was studied separately in class.
- With more time, I would replace the manual 1–5 quality rubric with an automated check (e.g., running generated code against unit tests) to remove scoring subjectivity, and would test the same modules against a second LLM provider to see whether the strategy recommendations in Section 6 still hold.
- The weighted decision matrix in Section 6 was the most useful part of the assignment for me personally – turning “which strategy is better” from a gut feeling into a number that changes predictably when the weights change made the trade-off between quality, latency, and cost much easier to explain to someone else.
- The hardest part was designing a rubric (Correctness/Completeness/Clarity) that was strict enough to be meaningful but simple enough to apply consistently by hand across 27 individual runs (3 modules × 3 strategies × 3 repeats); this experience made the case for automated scoring in future iterations very concrete rather than theoretical.

10. References

- T. B. Brown et al., “Language Models are Few-Shot Learners,” in Proc. NeurIPS, 2020.

- J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in Proc. NeurIPS, 2022.
- OpenAI, “OpenAI API Documentation,” 2024. [Online]. Available: platform.openai.com/docs.
- Anthropic, “Claude API Documentation,” 2024. [Online]. Available: docs.anthropic.com.
- Streamlit Inc., “Streamlit Documentation,” 2024. [Online]. Available: docs.streamlit.io.
- OpenAI, “tiktoken – Fast BPE Tokeniser for OpenAI Models,” GitHub repository, 2024.
- P. Liu et al., “Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in NLP,” ACM Computing Surveys, 2023.
- N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” Trans. ACL, 2024.
- McKinsey & Company, “The Economic Potential of Generative AI,” 2023. [Online]. Available: mckinsey.com.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Prompt Engineering, LLM Integration, Evaluation)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Python, LLM APIs, Prompt Engineering Frameworks)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Prompt Engineering	CO1/CO2	PO1, PO2, PO3	L3/L4	8

Techniques				
Application – LLM Task Module Integration	CO1	PO1, PO2, PO3	L3/L4	6
Application – Evaluation & Comparison of Prompts	CO2	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated PromptCraft	CO1–CO2	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Python, LLM APIs, Prompt Engineering)	CO1–CO2	PO5	L3	10
Validation of results	CO1–CO2	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO2	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO2	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA08 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____